# Ahmed GHALEB

## Al-Muneer Hall Reservations by Ahmed Ghaleb- PSM 1-45-59

### Document Details

Submission ID
trn:oid:::3618:144787863

Submission Date
Jul 2, 2026, 11:03 PM GMT+0

Download Date
Jul 3, 2026, 1:52 AM GMT+0

File Name
Al-Muneer Hall Reservations by Ahmed Ghaleb- PSM 1-45-59.pdf

File Size
2.3 MB

15 Pages

1,879 Words

10,433 Characters

# *% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

**Disclaimer**

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

# CHAPTER 5

# RESULTS, TESTING AND DISCUSSION

## 5.1 Introduction

The following chapter highlights the implementation results of the Al-Muneer Online Portal after shifting from the PSM1 requirement design phase to the PSM2 implementation phase. The portal was developed using the Spring Boot framework and PostgreSQL database, authenticated with JWT and developed using Thymeleaf on the server side along with vanilla JS and Chart.js. The implementation process has been done using the Waterfall model which has been discussed in Chapter 3 while the requirements and design artifacts have been used from Chapter 3 and 4 respectively.

Discussion of the topic will be split into three major parts. The section 5.2 describes the process of coding of system major functionalities according to the changelog of the system implemented versions v0.3-v0.7. In section 5.3 there will be listed the key user interfaces, which show the results achieved by the system, including placeholders of screenshot descriptions. In section 5.4 there is described the process of testing performed, which includes black box testing, white box testing and user testing.

## 5.2 Coding of System's Main Functions

Backend for Al-Muneer Online Portal was built in Java 21 programming language using Spring Boot 3.4.4 framework, where the build tool is Maven. PostgreSQL 16 relational database system was used, and the security was provided using Spring Security framework with JWT token and BCrypt algorithm hashing. The frontend was created using HTML5, CSS3, Thymeleaf template engine, and plain JavaScript, with Chart.js library for visualisation of data. This section provides information about the main functional modules that have been programmed according to iterative changelog from v0.3 to v0.7.

### 5.2.1 Backend Foundation and Security

The backend architecture is a layered architecture: controllers, services, repositories and JPA entities . `AdminAuthController` is a controller to login user through Spring Security framework. The JWT tokens are stored in HTTP-only cookies for an authenticated session. Passwords are encrypted with BCrypt and the DataSeeder creates a default admin account during first start-up of the application. Domain classes are: `BookingInquiry`, `PaymentProof`, `AvailabilitySlot`, `PricingTier`, `Media`, `Feedback`, `PageVisit`, and `NotificationTemplate`.

AdminAuthController.java

```java
@Controller
@RequestMapping("/admin")
@RequiredArgsConstructor
public class AdminAuthController {

    private final AuthenticationManager authenticationManager;
    :
    @PostMapping("/login")
    public String login(@RequestParam String username,
                        @RequestParam String password,
                        HttpServletResponse response,
                        Model model) {
        try {
            Authentication auth = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(username, password));

            UserDetails userDetails = (UserDetails) auth.getPrincipal();
            String token = jwtUtil.generateToken(userDetails);

            Cookie cookie = new Cookie("jwt", token);
            cookie.setHttpOnly(true);
            cookie.setPath("/");
            cookie.setMaxAge(86400); // 24 hours
            response.addCookie(cookie);

            return "redirect:/admin/dashboard";
        } catch (AuthenticationException e) {
            model.addAttribute("error", "Invalid username or password");
            return "admin/login";
        }
    }
```

Figure 5.1: Code Snippet: Admin Login and JWT Cookie Issuance

### 5.2.2 Visitor Booking Flow and Reference Code System

The implementation's main achievement is the uniform visitor booking process. The homepage has been converted into a one page scrolling landing page with anchor links. The hero section, venue details, media section, availability calendar and pricing packs have all been condensed onto a single page. Visitors interact through the interactive calendar where the future dates are with "Available" status and past dates are greyed out. The button "Submit inquiry" when clicking on an available date submits the date to `/inquiry?date=YYYY-MM-DD`.

The inquiry landing page has two purposes, add an inquiry and show an existing inquiry based on a reference number. The reference number of each inquiry is generated randomly and saved as a 9-character `referenceCode`, which is stored in a 150-day long HTTP-only cookie with the name `inq_ref`. In addition to this, visitors can even cancel their inquiries on the confirmation page without any evidence of payment.

BookingInquiry.java

```
14      @Id
15      @GeneratedValue(strategy = GenerationType.IDENTITY)
16      private Long inquiryId;
17
18      /**
19       * Visitor-facing 9-digit reference code (e.g. 472918305).
20       * Random, unique — used for lookup and cookie storage.
21       * Nullable only to allow Hibernate schema-update on existing tables;
22       * @PrePersist always populates this before insert.
23       */
24      @Column(unique = true)
25      private Long referenceCode;
        ⋮
58      @PrePersist
59      protected void onCreate() {
60          this.submissionDate = LocalDateTime.now();
61          if (this.status == null) {
62              this.status = InquiryStatus.NEW;
63          }
64          if (this.referenceCode == null) {
65              // 9-digit random: 100_000_000 - 999_999_999
66              this.referenceCode = ThreadLocalRandom.current().nextLong(100_000_000L, 1_000_000_000L);
67          }
68      }
```

Figure 5.2: Code Snippet: Booking Inquiry Submission Flow

### 5.2.3 Payment Proof, Status Cascade, and Notifications

The payment proof upload feature enables the visitor to submit their proof of offline payments in the form of images. The files are stored by the `FileUploadUtil`

class in the `uploads/proofs/` directory while the file name in UUID format is saved in the `PaymentProof` class. Upon verifying the proof, an atomic cascade update is performed, making the payment proof Verified, and the `BookingInquiry` CONFIRMED along with `AvailabilitySlot` BOOKED.

WhatsApp notifications can be delivered dynamically using template-based systems. The system provides administrators the ability to add, edit and remove the notification templates within the admin section. Within the inquiry/payment detail view, the admin can use a dropdown list to select the notification template, preview the notification message with the placeholders replaced, and also generate the URL client-side for `wa.me`.

PaymentProofServiceImpl.java

```java
    @Override
    @Transactional
    public PaymentProof updateStatus(Long proofId, VerificationStatus status, String adminNotes) {
        PaymentProof proof = getById(proofId);
        proof.setVerificationStatus(status);
        if (adminNotes != null && !adminNotes.isBlank()) {
            proof.setAdminNotes(adminNotes);
        }
        PaymentProof saved = proofRepository.save(proof);

        // Cascade: if Verified → Inquiry=CONFIRMED, Slot=BOOKED
        if (status == VerificationStatus.VERIFIED) {
            BookingInquiry inquiry = saved.getInquiry();
            inquiry.setStatus(InquiryStatus.CONFIRMED);
            inquiryRepository.save(inquiry);

            if (inquiry.getSlot() != null) {
                inquiry.getSlot().setStatus(SlotStatus.BOOKED);
                slotRepository.save(inquiry.getSlot());
            }
        }

        return saved;
    }
```

Figure 5.3: Code Snippet: Payment Verification Status Cascade

### 5.2.4 Administrator Modules and Analytics

The admin dashboard offers a daily workload snapshot in terms of new and active inquiries, pending payment proofs, unreviewed feedback, today's and past seven days' visits, average feedback rating, recent inquiries, and popular pages. Analytics shows the website traffic statistics by means of Chart.js. There is a bar chart for popular pages and line chart for daily trends of the past 30 days on the analytics page. Reports offer pie and bar charts to show the statuses of the inquiries,

payments, and feedback ratings within the chosen time period by means of `?fromDate=` and `?toDate=` query parameters.

AdminDashboardService.java

```java
public DashboardStats getStats() {
    long newInquiries = inquiryRepository.countByStatus(InquiryStatus.NEW);
    long activeInquiries = inquiryRepository.countByStatusNotIn(
            List.of(InquiryStatus.COMPLETED, InquiryStatus.CANCELLED_BY_ADMIN, InquiryStatus.CANCELLED_BY_VISITOR));

    long pendingPayments = paymentProofRepository.countByVerificationStatus(
            VerificationStatus.PENDING_VERIFICATION);

    long unreviewedFeedback = feedbackRepository.countByIsReviewed(false);

    LocalDate today = LocalDate.now();
    long visitsToday = sumHitsForDay(today);
    long visitsLast7Days = sumHitsSince(today.minusDays(6));

    double avgRating = feedbackRepository.findAverageRating();

    List<BookingInquiry> recent = inquiryRepository.findAllByOrderBySubmissionDateDesc()
            .stream().limit(5).toList();

    Map<String, Long> topPages = new LinkedHashMap<>();
    pageVisitRepository.findTotalHitsPerPage().stream().limit(5).forEach(row ->
            topPages.put((String) row[0], ((Number) row[1]).longValue()));

    return new DashboardStats(
            newInquiries,
            activeInquiries,
            pendingPayments,
            unreviewedFeedback,
            visitsLast7Days,
            visitsToday,
            avgRating,
            recent,
            topPages
    );
}
```

Figure 5.4: Code Snippet: Admin Dashboard Statistics Aggregation

### 5.2.5 AI Integration with Gemini

Version v0.7 added three non-blocking AI advisors powered by the Gemini model and the `google-genai` Java SDK (version 1.56.0):

- **AI Business Report Advisor** derives booking conversion and cancellation rates and prompts Gemini for three number backed action bullets.
- **AI Feedback Advisor** is separating low rating complaints from positive ones so the owner should address first.
- **AI Traffic Funnel Advisor** maps Home → Gallery → Pricing → Inquiry visit counts to see funnel drop off and suggest concrete conversion improvement.

All the AI panels are asynchronously loaded via dedicated `GET /ai-insight` endpoints after the page is rendered, so the main interface remains fast and can degrade gracefully if the API is unavailable.

## 5.3 Essential Interfaces Showing System's Results and Achievements

This section describes the main interfaces that demonstrate the functional aspects of the developed solution. Screenshots from the actual system are used to fill the placeholders in the final report.

### 5.3.1 Visitor Home Page and Booking Calendar

Visitor home page has one page scroll design. First up is the hero section, then the venue info which includes a Google Maps iframe, a masonry gallery with filters based on labels, and an availability calendar and pricing packages. Also there is anchor navigation with scroll spy to highlight the current section. The calendar allows the user to change months, shows the difference between available and booked days and activates a context menu “Submit inquiry” when an available day is clicked by the visitor.

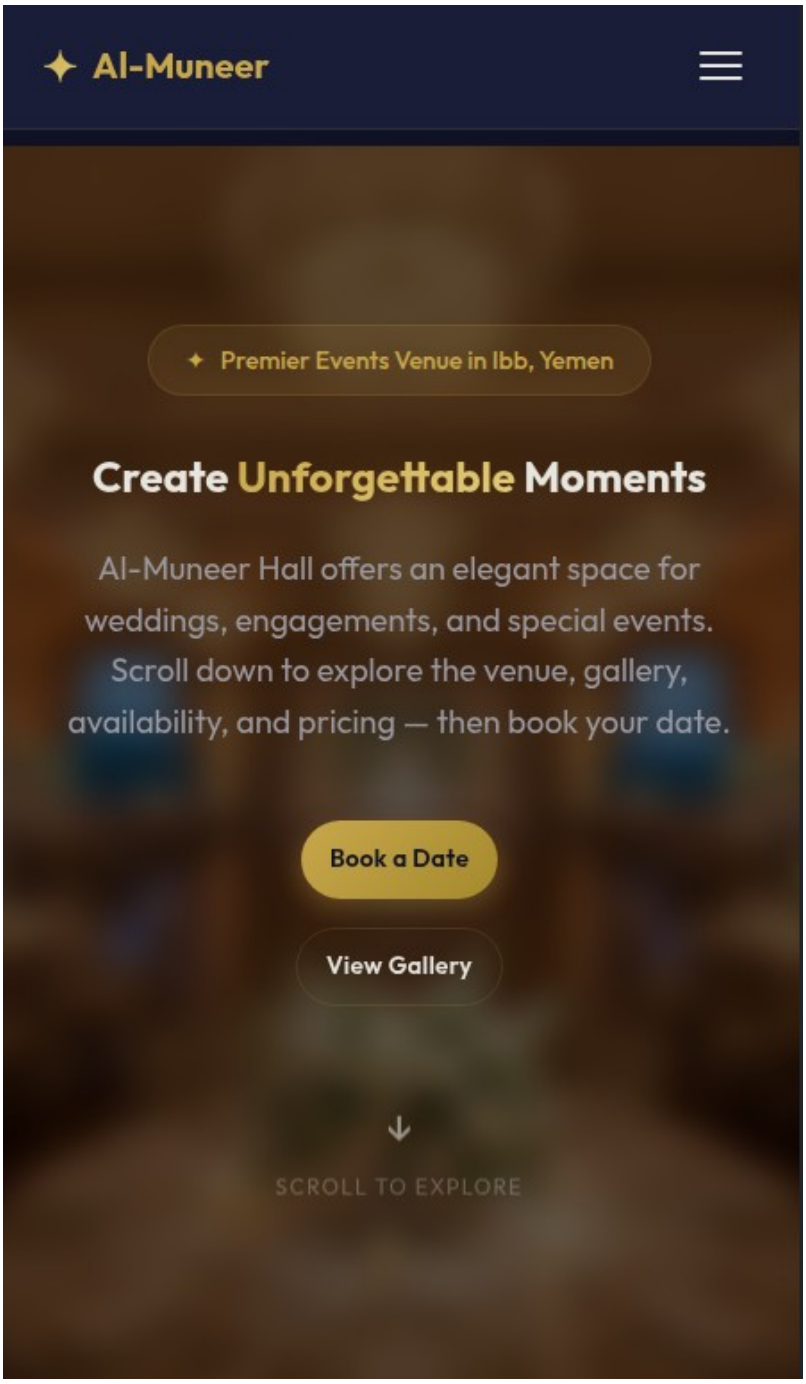


**Figure 5.5:** Visitor Home Page with Single-Page Scroll Layout

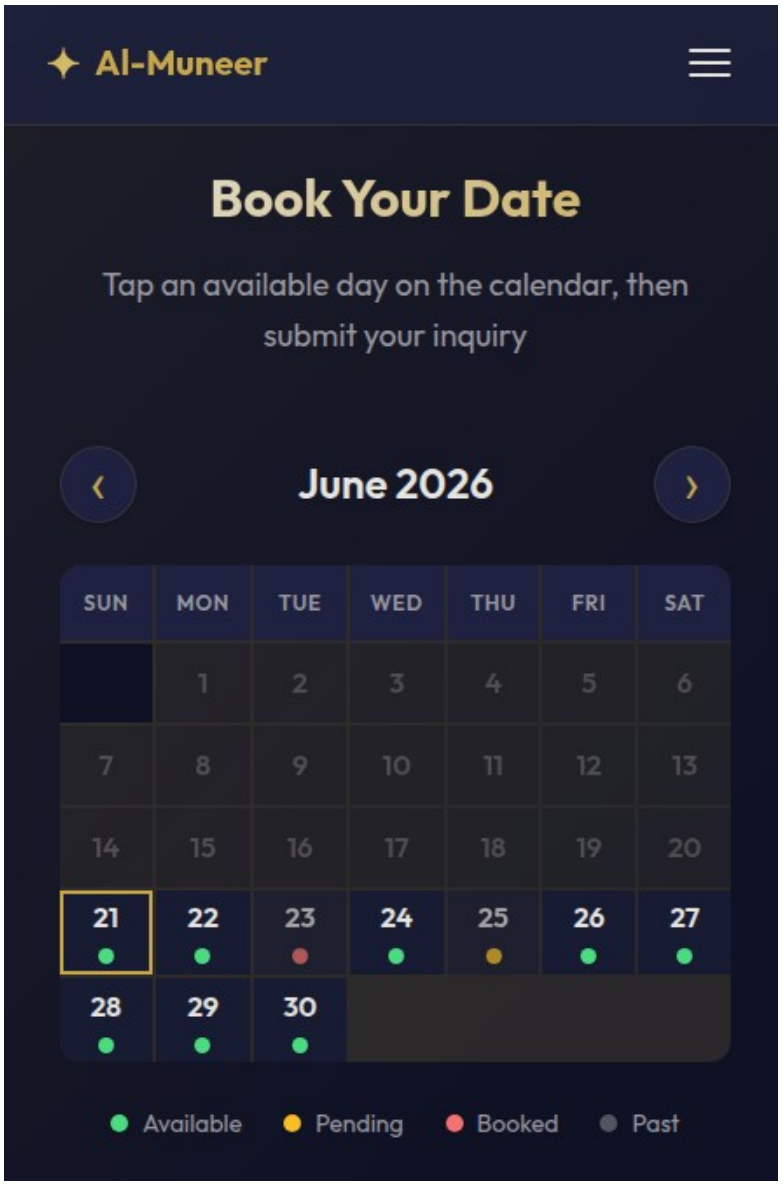


**Figure 5.6:** Visitor Availability Calendar and Inquiry CTA

### 5.3.2 Inquiry Form and Reference Code Lookup

The /inquiry page serves both new enquiries as well as lookup enquiries. When accessed from the homepage, the user is given prefilled fields in the form for the date and pricing package. The visitor receives a confirmation page with the 9-digit reference code and the option to cancel the inquiry themselves after submitting the form. The same page has a lookup field for the reference code.

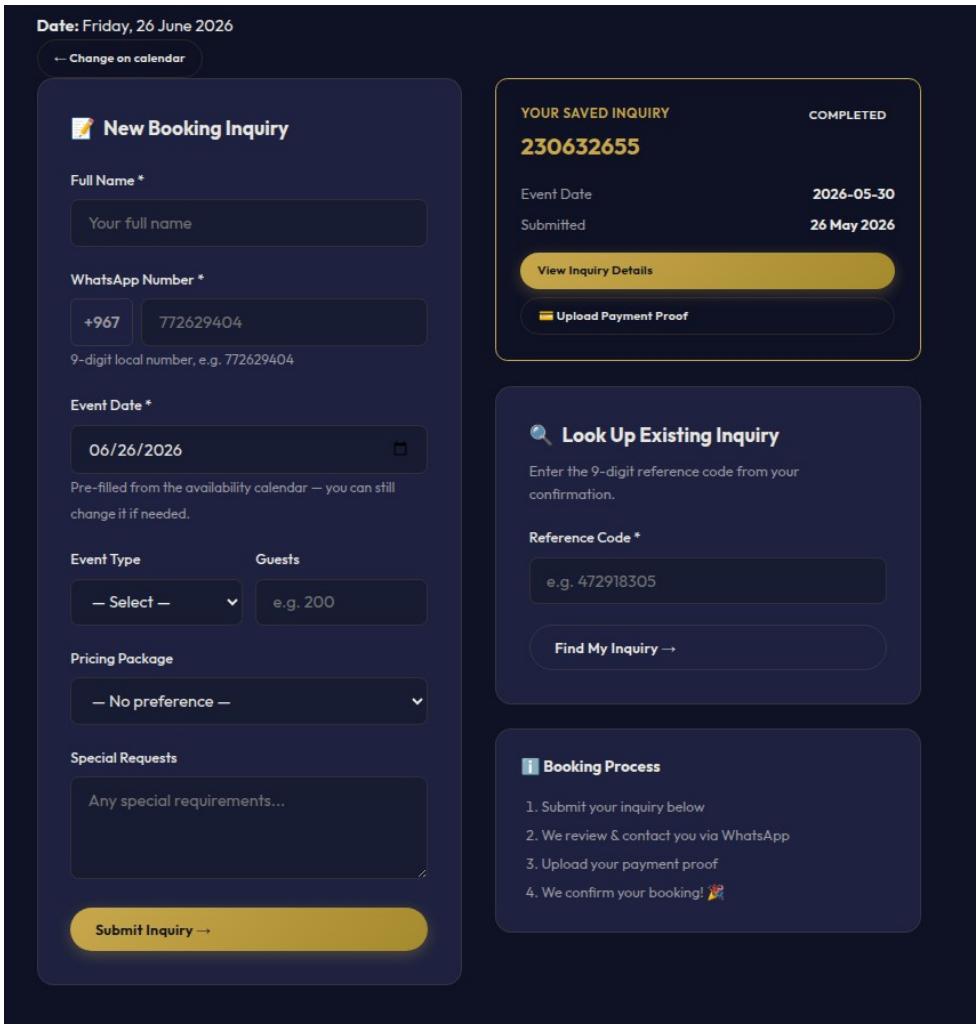


Figure 5.7: Inquiry Form with Pre-filled Date and Package

### 5.3.3 Admin Dashboard and Analytics

The admin dashboard provides an overview of the day-to-day business of the company to the owner. These are pending/active requests, proof pending, feedback not reviewed, and traffic history. The analytics page is further complemented by visualisation through the Chart.js graphs and AI Traffic Funnel Advisor.

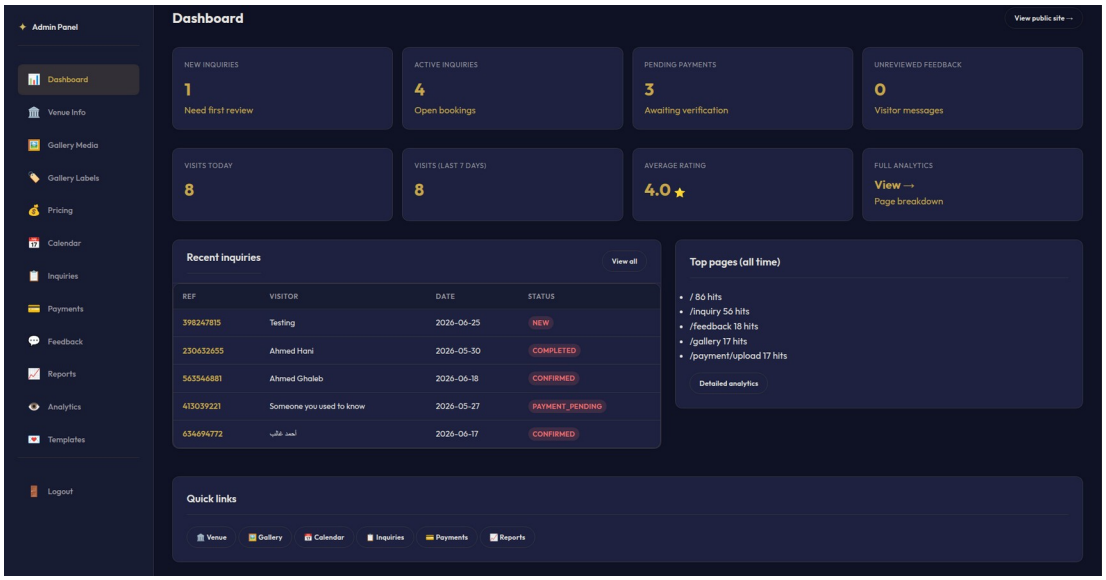


Figure 5.8: Administrator Dashboard Overview

### 5.3.4 Admin Inquiry and Payment Management

The status filter chip along with per-status count, client-side search and “only active” default view is part of the inquiry management panel. The information shown in each row includes the visitor’s reference code, contact details, date of the event, and the status of the request. In the detail view, the administrator has options for updating status, viewing the attached payment proofs relevant to that inquiry, and sending a dynamic WhatsApp message from a list of templates.

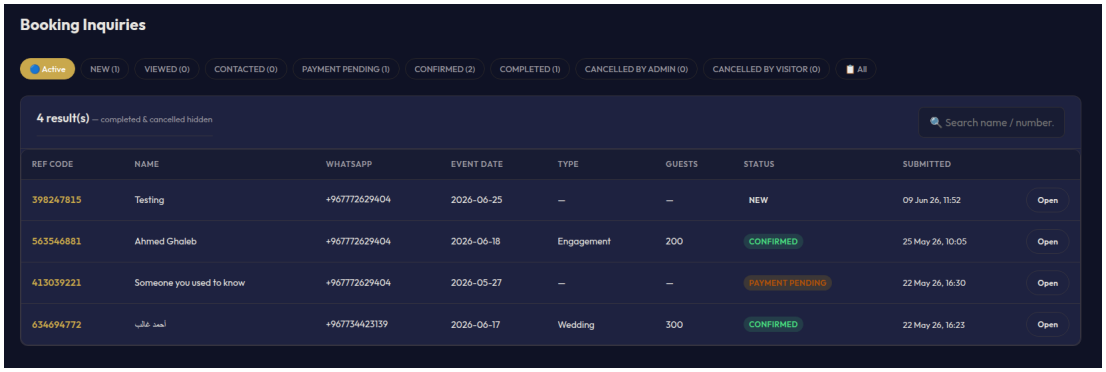


Figure 5.9: Administrator Inquiry Management Panel

### 5.3.5 Reports with Visualisations

The Reports section depicts operational information using visual diagrams. The pie charts reveal information about inquiry status and payment status. A bar chart reveals information on feedback ratings. Users can use the date filter to view the trends during a certain period. The AI Business Report Advisor section gets loaded asynchronously for useful suggestions on the basis of current report information.

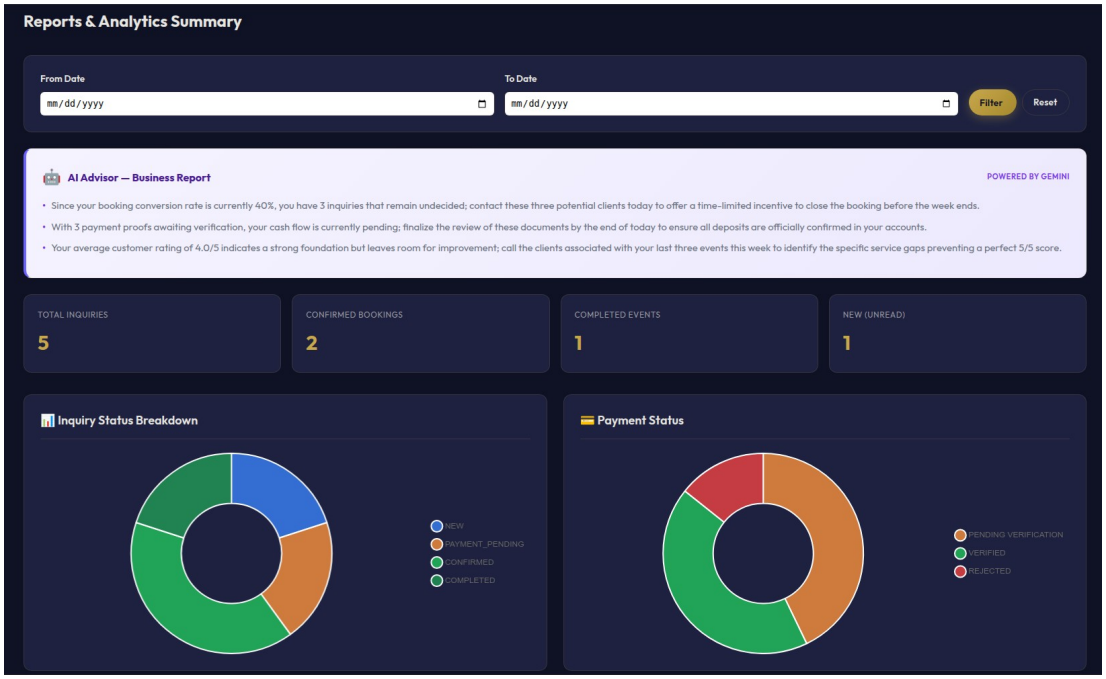


Figure 5.10: Reports Page with Visual Charts


Figure 5.11: Analytics Dashboard with Chart.js Visualizations

## 5.4 Testing

Testing is carried out during the implementation stage to ensure that the Al-Muneer Online Portal satisfies the functional and non-functional requirements stated in the SRS. The methods used included black box, white box, and user testing.

### 5.4.1 Black Box Testing

The black-box testing was concerned with flows within the system, input/output behavior and errors without looking into the source code. The test cases have been generated using the use cases in the SRS document and changelog features:

- **Visitor flow:** Submission of valid/invalid inquiry form, pre-fill date/package information from homepage, search for inquiries through reference number, and cancellation of inquiries.
- **Payment proof flow:** Uploading valid and invalid file types, verifying a proof as an admin, and confirming the status cascade to inquiry and slot.
- **Admin flow:** Login, management of venues/gallery label names, updates to calendar slots, inquiry filtration, messaging through WhatsApp templates, and report generation.
- Error handling: Invalid login credentials, missing required fields, oversized uploads, and unauthorized access attempts all produced clear user-friendly messages.

The black box tests confirmed that the system behaves according to specification and handles edge cases gracefully.

### 5.4.2 White Box Testing

White box testing was used to test the internal logic and code paths. The following components were unit and integrated tested:

- Reference code generation: We confirmed that a unique 9-digit code is assigned to `BookingInquiry`.
- WhatsApp normalisation: `normaliseWhatsApp()` is known to remove non-digits and prefix the configured country code.
- Status cascade: Confirmed that when a `PaymentProof` is checked, the corresponding `BookingInquiry` is set to CONFIRMED and the `AvailabilitySlot` is set to BOOKED.
- JWT security: Token creation, validation and expiration management.

**AI service fallback:** Verified `GeminiService` provides a graceful fallback message on API failure without crashing the page.

confirmed that the core business rules and security mechanisms are working correctly at the code level.

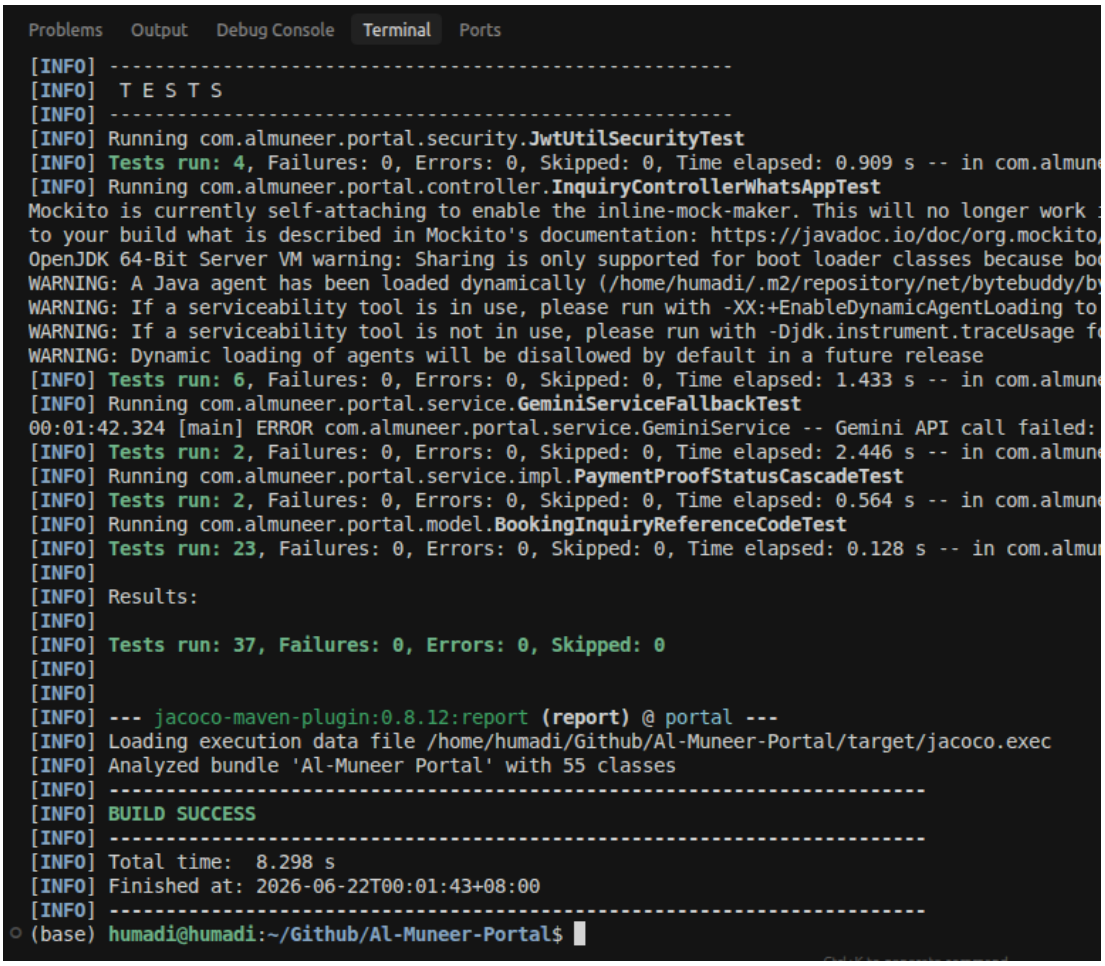


Figure 5.9: Unit Testing Log

### 5.4.3 User Testing

Project stakeholder Mr. Ahmed Almunajid owner of Al-Muneer Hall participated in user testing. The owner performed representative duties for the website such as updating venue information, uploading gallery pictures, looking at new enquiries, looking at a payment proof, and looking at reports. Interface clarity, workflow efficiency and usefulness of the AI advisors were collected as feedback. Minor changes to dashboard card ordering and notification template defaults were made based on this feedback.

## 5.5 Chapter Summary

This chapter presents the results of the implementation of the Al-Muneer Online Portal. Section 5.2 highlighted the evolution of core functions like the Spring Boot back end, visitor reservation process using codes, cascade of payment evidence verification, automated notifications via WhatsApp, dashboards for administrators, analytical data, reports, and Gemini AI advisors. Section 5.3 described the critical interfaces and Section 5.4 discussed the black-box, white-box and user testing. The system developed clearly satisfies all requirements in the SRS and provides a practical approach.